

Code-review

(рецензирование кода)

Code-review (рецензирование кода) —

это важнейшая часть профессиональной разработки, особенно в контексте ПМ.05 (Проектирование и разработка ИС). На практическом занятии по планированию code-review вы должны научиться организовывать процесс проверки кода, составлять чек-листы и анализировать чужой код.

Цель занятия:

Научиться планировать
процесс code-review,
составлять критерии
проверки кода и находить
ошибки/недочёты в чужом
коде.

План проведения code-review

1. Общие положения

Code-review (рецензирование кода) — процесс проверки исходного кода автора другими участниками команды с целью:

- ▶ выявления ошибок и уязвимостей;
- ▶ улучшения читаемости и поддерживаемости кода;
- ▶ соблюдения стандартов кодирования;
- ▶ обмена знаниями между разработчиками.

2. Участники процесса

Роль	Ответственность
Автор	Пишет код, создаёт запрос на проверку (Pull Request/Merge Request), исправляет замечания
Рецензент (1-2 человека)	Проверяет код по чек-листу, оставляет комментарии, утверждает изменения
Технический лид (опционально)	Разрешает спорные ситуации, утверждает критически важные изменения

3. Инструменты

Инструмент	Назначение
GitHub / GitLab	Хостинг репозитория, Pull Requests, обсуждения
Flake8 / pylint	Автоматическая проверка стиля кода (PEP 8)
mypy	Проверка аннотаций типов
pytest	Запуск автоматических тестов
SonarLint / Qodana	Дополнительный статический анализ
Telegram / Slack	Уведомления о новых проверках

4. Критерии проверки (чек-лист)

4.1. Функциональность и логика

- ▶ Код решает поставленную задачу?
- ▶ Нет логических ошибок, бесконечных циклов, неверных условий?
- ▶ Корректно обрабатываются граничные случаи (пустые списки, нули, None)?

4.2. Обработка ошибок

- ▶ Проверяются ли входные данные?
- ▶ Обрабатываются ли исключения (try-except)?
- ▶ Нет ли скрытых ошибок (деление на ноль, выход за границы списка)?

4.3. Стиль и читаемость

- ▶ Код соответствует PEP 8 (отступы, пробелы, длина строк)?
- ▶ Имена переменных, функций, классов понятны и отражают суть?
- ▶ Нет «магических чисел» — используются именованные константы?

4.4. Документация

- ▶ Есть ли docstring у каждой функции/класса?
- ▶ Сложные участки кода снабжены комментариями?
- ▶ Есть ли общая документация по проекту (README)?

4.5. Архитектура и дизайн

- ▶ Соблюдается ли принцип **DRY** (Don't Repeat Yourself)?
- ▶ Функции слишком длинные? (рекомендуется не более 30-50 строк)
- ▶ Код разбит на логические модули/классы?

4.6. Производительность

- ▶ Нет неэффективных алгоритмов (вложенные циклы там, где можно обойтись словарём)?
- ▶ Ресурсы (файлы, соединения с БД) закрываются?
- ▶ Нет лишних запросов к БД или API?

4.7. Безопасность

- ▶ Нет уязвимостей (SQL-инъекции, XSS, открытые пароли в коде)?
- ▶ Используется ли параметризованные запросы вместо конкатенации строк?
- ▶ Валидируются ли данные от пользователя?

4.8. Тестирование

- ▶ Написаны ли **unit-тесты** для ключевых функций?
- ▶ Проходят ли все существующие тесты после изменений?
- ▶ Есть ли тесты на граничные случаи?

4.9. Типизация

- ▶ Используются ли аннотации типов (type hints)?
- ▶ Аннотации соответствуют реальным типам данных?

Пример оформления в дневнике-отчёте по практике

Мною был разработан и внедрён план проведения code-review в команде из 4 человек. В качестве инструментов использовались GitHub Pull Requests, автоматические линтеры Flake8 и mypy. За время практики было проведено 8 проверок кода, в ходе которых выявлено 12 критических и 24 некритических замечания. Все замечания были устранены, качество кода повысилось, количество багов в продакшене снизилось на 30%.

Подпись студента: _____ / Дата: _____

Подпись руководителя практики: _____